

# 基本挿入法（挿入ソート）

「基本交換法（バブルソート）」「基本選択法（選択ソート）」と並んで、基本3種の中で最も実践的なのが、この「基本挿入法（挿入ソート）」です。

挿入ソートは、私たちが「手持ちのトランプを並べ替えるときの動き」に最も近いアルゴリズムです。

## 1. 挿入ソートのイメージ

バラバラに配られたトランプを左から順に手に取って、並べ替える場面を想像してください。

- 1. 左端の1枚を「整列済み」とみなします。
- 2. 次に、まだ見ていないカード（右側）から1枚取り出します。
- 3. そのカードを、すでに並んでいるカードの中に「正しい位置はどこかな？」と探して差し込み（挿入）します。
- 4. これを最後の1枚まで繰り返します。

## 2. 具体的な手順（4, 2, 5, 1 を並べ替える）

例によって、この数列を下記のように図示します。

| Status | 比較対象 | 数値 | 図     |
|--------|------|----|-------|
|        |      | 4  | ####  |
|        |      | 2  | ##    |
|        |      | 5  | ##### |
|        |      | 1  | #     |

「整列済みのグループ」が左からどんどん伸びていくイメージで進めます。

### 【1周目】「2」をどこに入れるか決める

- [4, | 2, 5, 1]：最初の「4」は整列済みと考えます。

| Status  | 比較対象 | 数値 | 図        |
|---------|------|----|----------|
| 整列済みとする |      | 4  | \$\$\$\$ |
|         |      | 2  | ##       |
|         |      | 5  | #####    |
|         |      | 1  | #        |

- 次の「2」を取り出します。

| Status   | 比較対象 | 数値 | 図        |
|----------|------|----|----------|
| いったん取り出す |      | 2  | ##       |
| Status   | 比較対象 | 数値 | 図        |
| 整列済みとする  |      | 4  | \$\$\$\$ |
|          |      | 5  | #####    |
|          |      | 1  | #        |

- 「4」と比べる

| Status   | 比較対象 | 数値 | 図        |
|----------|------|----|----------|
| 「4」と比較する | *    | 2  | ##       |
| Status   | 比較対象 | 数値 | 図        |
| 整列済みとする  | *    | 4  | \$\$\$\$ |
|          |      | 5  | #####    |
|          |      | 1  | #        |

「4」と比較すると「2」の方が小さいので、「4」の前に**挿入**します。

| Status   | 比較対象 | 数値 | 図        |
|----------|------|----|----------|
| 「4」の前に挿入 | *    | 2  | ##       |
| 整列済みとする  | *    | 4  | \$\$\$\$ |
|          |      | 5  | #####    |
|          |      | 1  | #        |

- [2, 4, | 5, 1] → 左側2枚が整列済み。

| Status  | 比較対象 | 数値 | 図        |
|---------|------|----|----------|
| 整列済みとする |      | 2  | \$\$     |
| 整列済みとする |      | 4  | \$\$\$\$ |
|         |      | 5  | #####    |
|         |      | 1  | #        |

## 【2周目】「5」をどこに入れるか決める

- 次の「5」を取り出します。

| Status  | 比較対象 | 数値 | 図     |
|---------|------|----|-------|
| 挿入先を決める |      | 5  | ##### |

| Status  | 比較対象 | 数値 | 図        |
|---------|------|----|----------|
| 整列済みとする |      | 2  | \$\$     |
| 整列済みとする |      | 4  | \$\$\$\$ |
|         |      | 1  | #        |

- 整列済みの下端の「4」と比べます。

| Status   | 比較対象 | 数値 | 図        |
|----------|------|----|----------|
| 「4」と比較する | *    | 5  | #####    |
| Status   | 比較対象 | 数値 | 図        |
| 整列済みとする  |      | 2  | \$\$     |
| 整列済みとする  | *    | 4  | \$\$\$\$ |
|          |      | 1  | #        |

- 「5」の方が大きいので、そのまま（4の後ろに）**挿入**します。

| Status   | 比較対象 | 数値 | 図        |
|----------|------|----|----------|
| 整列済みとする  |      | 2  | \$\$     |
| 整列済みとする  | *    | 4  | \$\$\$\$ |
| 「4」の後に挿入 | *    | 5  | #####    |
|          |      | 1  | #        |

- [2, 4, 5, | 1] → 左側3枚が整列済み。

| Status  | 比較対象 | 数値 | 図          |
|---------|------|----|------------|
| 整列済みとする |      | 2  | \$\$       |
| 整列済みとする |      | 4  | \$\$\$\$   |
| 整列済みとする |      | 5  | \$\$\$\$\$ |
|         |      | 1  | #          |

### 【3周目】「1」をどこに入れるか決める

- 最後の「1」を取り出します。

| Status  | 比較対象 | 数値 | 図    |
|---------|------|----|------|
| 挿入先を決める |      | 1  | #    |
| Status  | 比較対象 | 数値 | 図    |
| 整列済みとする |      | 2  | \$\$ |

| Status  | 比較対象 | 数値 | 図          |
|---------|------|----|------------|
| 整列済みとする |      | 4  | \$\$\$\$   |
| 整列済みとする |      | 5  | \$\$\$\$\$ |

- 整列済みの下端の「5」と比較すると、「5」より小さい、とわかります。

| Status   | 比較対象 | 数値 | 図 |
|----------|------|----|---|
| 「5」と比較する | *    | 1  | # |

| Status  | 比較対象 | 数値 | 図          |
|---------|------|----|------------|
| 整列済みとする |      | 2  | \$\$       |
| 整列済みとする |      | 4  | \$\$\$\$   |
| 整列済みとする | *    | 5  | \$\$\$\$\$ |

- 次の「4」と比較すると、「4」より小さい、とわかります。

| Status   | 比較対象 | 数値 | 図 |
|----------|------|----|---|
| 「4」と比較する | *    | 1  | # |

| Status  | 比較対象 | 数値 | 図          |
|---------|------|----|------------|
| 整列済みとする |      | 2  | \$\$       |
| 整列済みとする | *    | 4  | \$\$\$\$   |
| 整列済みとする |      | 5  | \$\$\$\$\$ |

- 次の「2」と比較すると、「2」より小さい、とわかります。

| Status   | 比較対象 | 数値 | 図 |
|----------|------|----|---|
| 「2」と比較する | *    | 1  | # |

| Status  | 比較対象 | 数値 | 図          |
|---------|------|----|------------|
| 整列済みとする | *    | 2  | \$\$       |
| 整列済みとする |      | 4  | \$\$\$\$   |
| 整列済みとする |      | 5  | \$\$\$\$\$ |

- 他の数より小さいので、一番先頭に **挿入** します。

| Status  | 比較対象 | 数値 | 図        |
|---------|------|----|----------|
| 先頭に挿入する | *    | 1  | #        |
| 整列済みとする | *    | 2  | \$\$     |
| 整列済みとする |      | 4  | \$\$\$\$ |

| Status  | 比較対象 | 数値 | 図        |
|---------|------|----|----------|
| 整列済みとする |      | 5  | \$\$\$\$ |

- [1, 2, 4, 5] → 完成！

| Status  | 比較対象 | 数値 | 図        |
|---------|------|----|----------|
| 整列済みとする |      | 1  | \$       |
| 整列済みとする |      | 2  | \$       |
| 整列済みとする |      | 4  | \$\$\$   |
| 整列済みとする |      | 5  | \$\$\$\$ |

### 3. 挿入ソートのすごいところ（メリット）

実は、基本3種（交換・選択・挿入）の中で、挿入ソートには特筆すべき強みがあります。

- 「だいたい並んでいるデータ」にめっちゃくちゃ強い  
すでに順番が整っているデータなら、「どこに挿入しようかな？」と探す手間がほとんど省けるため、バブルソートや選択ソートよりもずっと速く終わります。
- オンライン処理ができる  
データが後から1つずつ追加されても、その都度「正しい場所」に差し込むだけでいいので、リアルタイムな処理に向いています。

### 4. 特徴のまとめ

| 項目    | 特徴                                   |
|-------|--------------------------------------|
| 考え方   | 取り出した値を、整列済みの列の「適切な位置」に差し込む。         |
| メリット  | 元のデータがある程度並んでいる場合、計算時間が非常に短い。        |
| デメリット | 逆順（大きい順）に並んでいるデータを並べ替えるときは、移動が多くて大変。 |

これで「整列の基本3種」がすべて揃いましたね！

### 実装

「トランプを正しい位置に差し込む」動きをプログラムにした、挿入ソートのC言語実装を作成しました。

このプログラムの最大の特徴は、「今持っている値を、整列済みの列のどこに入れるべきか、後ろから順番に比較して隙間を作る」という点です。

### 疑似言語

2023年からの新形式に基づいた、挿入ソートの疑似言語実装です。このアルゴリズムは、整列済みの列に対して新しい値を適切な位置へ「挿入」していく手法です。

## 挿入ソート（新形式）の疑似言語実装

```

o insertionSort(整数型の配列: data)
  整数型: i, j, key, n
  n ← dataの要素数
  for (i を 2 から n まで 1 ずつ増やす)
    key ← data[i - 1] /* 挿入する値を一時保存 */
    j ← i - 1
    /* 自分より大きい要素を右にずらす */
    while (j ≥ 1 and data[j] > key)
      data[j] ← data[j - 1]
      j ← j - 1
    endwhile
    data[j + 1] ← key /* 空いた位置に値を挿入 */
  endfor

```

```

o insertionSort(整数型の配列: data)
  整数型: i, j, key, n
  n ← dataの要素数
  for (i を 1 から n - 1 まで 1 ずつ増やす)
    key ← data[i] /* 挿入する値を一時保存 */
    j ← i - 1
    /* 自分より大きい要素を右にずらす */
    while (j ≥ 0 and data[j] > key)
      data[j + 1] ← data[j]
      j ← j - 1
    endwhile
    data[j + 1] ← key /* 空いた位置に値を挿入 */
  endfor

```

### このコードのポイント：

- **効率性:** データがすでにある程度並んでいる場合、**while** ループの実行回数が減るため、非常に高速に動作します。
- **新形式のルール:** **while...endwhile** や **for...endfor** でブロックを明示し、配列の添字は「0」から開始しています。

## C言語による挿入ソートの実装

```

#include <stdio.h>

void insertionSort(int arr[], int n) {
    int i, j, key;

    // 2番目の要素（インデックス1）から順番に取り出す
    for (i = 1; i < n; i++) {
        key = arr[i]; // 現在注目している値（トランプから引いたカード）
        j = i - 1;

```

```

    /* 整列済みの部分（左側）を後ろから順に見ていく
       取り出した値（key）より大きい要素があれば、右に1つずらす
    */
    while (j >= 0 && arr[j] > key) {
        arr[j + 1] = arr[j]; // 右にずらす（コピーする）
        j = j - 1;
    }

    // 適切な位置が見つかったら、そこに値を差し込む
    arr[j + 1] = key;
}

int main() {
    int data[] = {4, 2, 5, 1}; // 説明で使った例
    int n = 4;

    printf("ソート前: ");
    for (int i = 0; i < n; i++) printf("%d ", data[i]);
    printf("\n");

    insertionSort(data, n);

    printf("ソート後: ");
    for (int i = 0; i < n; i++) printf("%d ", data[i]);
    printf("\n");

    return 0;
}

```

## ポイント

### 1. 変数 **key** の役割

- 差し込むための値を一時的に「手」に持っておく必要があります。これが **key** です。これをしておかないと、他の値を「ずらした」ときに元の値が上書きされて消えてしまいます。

### 2. **while** ループの条件

- arr[j] > key** という条件は、「自分より大きい先輩たちがいる間は、右にどいてもらう」という意味です。
- j >= 0** は、一番左端（先頭）まで到達した時に止めるための安全装置です。

### 3. ずらす（Shift）と挿入（Insert）

- arr[j + 1] = arr[j]** は、隣へのコピーです。これにより隙間が作られます。
- ループが終わった後の **arr[j + 1] = key** が、ようやく見つけた「正しい居場所」にカードを置く瞬間です。

もし、最初からデータがきれいに並んでいたら、このプログラムはどう動くと思いますか？

- **答え：** `while` ループの条件が最初から成立しないため、中身が一度も実行されません。
- **学び：** これにより、「挿入ソートは整列済みのデータに対して非常に高速である」という理論的な特徴を実感できたでしょう。